

Đề xuất Giai đoạn 2

GRAPES-GFN-Rec: học cách lấy mẫu đồ thị cho hệ gợi ý quy mô lớn dùng GNN

Ty Van Xuan

16 tháng 9 năm 2026

Tóm tắt nội dung

Giai đoạn 1 đã tái hiện GRAPES [1] — một phương pháp lấy mẫu đồ thị trong đó một GNN phụ học chọn node cho từng layer thông qua GFlowNet — trên bài toán phân loại node. Giai đoạn 2 chuyển ý tưởng này sang hệ gợi ý: đề xuất **GRAPES-GFN-Rec**, trong đó sampler học chọn node ngữ cảnh cho Sampled LightGCN [2] và được huấn luyện bằng Trajectory Balance [3] với phần thưởng từ BPR ranking loss [4]. Phương pháp được so sánh với lấy mẫu đều, lấy mẫu theo bậc, biến thể REINFORCE, Full LightGCN, MostPop và BPR-MF dưới cùng graph, budget, seed, evaluator và phần cứng. Dữ liệu chính là Amazon Reviews’23 Baby_Products [10]: graph huấn luyện có 3.868.654 cạnh, 2.318.308 user, 162.125 item, rất thưa và lệch long-tail. Báo cáo trình bày vấn đề, phân tích dữ liệu, thiết kế phương pháp, protocol đánh giá, rủi ro và kế hoạch triển khai theo tám gate. **Tại thời điểm viết, chưa có kết quả thực nghiệm nào của GRAPES-GFN-Rec**; phần đã hoàn thành là khóa thiết kế, lõi mã nguồn và bộ kiểm thử oracle.

Mục lục

1	Bối cảnh và động lực	3
1.1	GNN cho hệ gợi ý và bài toán bùng nổ lân cận	3
1.2	Lấy mẫu thay đổi thứ model học	3
1.3	Giai đoạn 1: GRAPES	3
1.4	Rà soát hướng đi trước giai đoạn 2	3
2	Câu hỏi nghiên cứu	4
3	Dữ liệu	4
3.1	Chọn dataset	4
3.2	Chất lượng và nhiễu	4
3.3	Mất cân bằng và long-tail	5
3.4	Đường đi dữ liệu và chia theo thời gian	5
4	Phương pháp đề xuất: GRAPES-GFN-Rec	6
4.1	Kiến trúc tổng thể	6
4.2	Chuyển đổi từ GRAPES sang gợi ý	6
4.3	Lấy mẫu theo layer	6
4.4	Sampled LightGCN	7
4.5	Hàm mục tiêu	7
4.6	Những điểm riêng của bài toán gợi ý	7
5	Thiết kế đánh giá	8
5.1	Phương pháp so sánh	8
5.2	Chỉ số	8
5.3	Tiêu chí kết luận	9

6	Tiến độ hiện tại	9
7	Kế hoạch triển khai	9
8	Rủi ro và phương án dự phòng	10
9	Quyết định cần ý kiến giảng viên	10

1 Bối cảnh và động lực

1.1 GNN cho hệ gợi ý và bài toán bùng nổ lân cận

Trong collaborative filtering dựa trên đồ thị, user và item là node, mỗi tương tác là một cạnh. LightGCN [2] học embedding bằng cách lan truyền và lấy trung bình embedding của hàng xóm qua L bước, không có ma trận biến đổi hay hàm phi tuyến. Để tính embedding của một user ở độ sâu L , model cần toàn bộ vùng lân cận L bước của user đó.

Kích thước vùng lân cận tăng theo tích các bậc gấp trên đường đi. Trên graph Baby_Products, một user trung bình có 1,67 item; nhưng khi đi theo một cạnh ngẫu nhiên, item gặp phải có bậc kỳ vọng

$$\mathbb{E}_{\text{cạnh}}[\deg(i)] = \frac{\sum_i \deg(i)^2}{\sum_i \deg(i)} \approx 1.009 \text{ user},$$

lớn hơn rất nhiều so với bậc trung bình 23,9 của item, vì cạnh dồn vào item phổ biến. Item lớn nhất có 21.348 user. Như vậy chỉ sau hai bước, một user có thể kéo theo hàng nghìn user khác; một batch nhiều triplet hợp các vùng này lại. Đây là hiện tượng *bùng nổ lân cận* (neighbourhood explosion) mà các phương pháp lấy mẫu như GraphSAGE [8], FastGCN [6], LADIES [7] và PinSage [9] tìm cách kiểm soát. Kích thước thực tế theo batch và budget sẽ được đo ở gate R3.

1.2 Lấy mẫu thay đổi thứ model học

Lấy mẫu giới hạn số node mỗi layer bằng một budget k . Đối lại, model chỉ nhìn thấy một phần ngữ cảnh. Vì vậy sampler không chỉ ảnh hưởng thời gian và bộ nhớ: nó quyết định tín hiệu collaborative mà recommender học được. Câu hỏi *chọn node nào* là câu hỏi nghiên cứu, không chỉ là chi tiết kỹ thuật.

1.3 Giai đoạn 1: GRAPES

GRAPES [1] dùng hai GNN: GCN_C giải bài toán chính và GCN_S chấm điểm hàng xóm để chọn đúng k node mỗi layer bằng Gumbel Top- k [5]. GCN_S được huấn luyện bằng GFlowNet Trajectory Balance hoặc REINFORCE, với chi phí là loss của GCN_C . Bảng 1 là kết quả tái hiện ở giai đoạn 1 trên Tesla T4.

Bảng 1: Tái hiện GRAPES ở giai đoạn 1 (accuracy, mean trên các run).

Dataset	GRAPES	Random	Thời gian GRAPES	Thời gian Random
Cora	0,8710	0,8677	56,3 s	30,5 s
Citeseer	0,7857	0,7900	94,6 s	56,0 s
ogbn-arxiv	0,6204	0,6128	11.114 s	8.133 s

Hai bài học: (i) sampler học được có thể tốt hơn chọn ngẫu nhiên nhưng lợi thế nhỏ và không đều giữa các dataset; (ii) sampler tốn thêm 37–85% thời gian, nên chi phí sampler phải nằm trong phép so sánh. GRAPES chỉ được đánh giá trên phân loại node; recommendation được nêu như hướng mở rộng nhưng chưa được kiểm chứng.

1.4 Rà soát hướng đi trước giai đoạn 2

Trước khi viết đề xuất này, hướng nghiên cứu đã được rà soát lại. Một nhánh pilot trước đó so sánh ba luật chọn node cố định (M0 đều, M1 theo bậc, M2 frontier-normalized). M2 mượn hình thức hành động của GRAPES (Gumbel Top- k , $K^l = V^0 \cup V^l$) nhưng thiếu phần cốt lõi: không có policy học được, không có log-likelihood trajectory, không có phần thưởng, không có mục tiêu TB/REINFORCE và không có bước cập nhật sampler. Vì vậy pilot không trả lời được câu hỏi của đề tài. Quyết định được ghi trong decision log (DL-001): giai đoạn 2 hiện thực một biến thể GRAPES học được, pilot chuyển vào phụ lục.

Để tránh lặp lại, một phương pháp chỉ được gọi là *biến thể GRAPES* nếu thỏa cả bảy điều kiện trong Bảng 2.

Bảng 2: Checklist G1–G7.

ID	Thành phần bắt buộc	M2 pilot
G1	Policy có tham số (GCN_S và embedding riêng của sampler)	không
G2	Hành động exact- k bằng Gumbel Top- k trên xác suất của policy	có (trên điểm cố định)
G3	Log-likelihood trajectory $\log q$	không
G4	Tín hiệu từ loss gợi ý, đã detach	không
G5	Normalizer $\log Z(V^0)$ học được	không
G6	Mục tiêu Trajectory Balance hoặc REINFORCE	không
G7	Tham số sampler thực sự thay đổi qua huấn luyện	không

2 Câu hỏi nghiên cứu

RQ1 (chính). Trên graph user–item implicit chia theo thời gian, khi giữ cố định recommender, graph, thứ tự triplet BPR, negative, số bước tối ưu, budget node mỗi layer, seed, evaluator và phần cứng, GRAPES-GFN-Rec có đạt trade-off NDCG@20–chi phí huấn luyện tốt hơn lấy mẫu đều và lấy mẫu theo bậc hay không?

RQ2. Trajectory Balance có ổn định hoặc hiệu quả hơn REINFORCE (GRAPES-RL-Rec) trên tín hiệu BPR không?

RQ3. Sampler học được chọn node nào (bậc, loại user/item, nhóm head/body/tail), và điều đó có khuếch đại popularity bias không?

RQ4. Lợi thế, nếu có, thay đổi thế nào khi budget k chặt hơn?

Kết quả âm là kết quả hợp lệ. Nghiên cứu hoàn thành khi trả lời các câu hỏi với bằng chứng truy vết được, không phụ thuộc vào việc GRAPES-GFN-Rec thắng.

3 Dữ liệu

3.1 Chọn dataset

Amazon Reviews’23 [10] có nhiều danh mục, số giao dịch lớn và metadata sản phẩm có thể nối thêm. Ba danh mục đã được kiểm toán chính xác trên file tải về:

Bảng 3: Portfolio dữ liệu Amazon đã kiểm toán.

Dataset	Dòng raw	Vai trò
All_Beauty	693.929	Kiểm tra pipeline; quá nhỏ, chỉ 3,98% target validation còn warm-start.
Baby_Products	5.953.891	Dữ liệu chính: 3,39 triệu user, 217.654 item; đủ lớn để thấy chi phí sampling, long-tail rõ, chạy được ma trận thí nghiệm trên Colab T4.
Home_and_Kitchen	66.623.880	Tham chiếu quy mô (11,2 lần Baby); dành cho stress test sau khi có kết quả Baby.

MovieLens 25M, Gowalla, Yelp2018, MIND và KuaiRec đã được cân nhắc. Chúng phù hợp các câu hỏi khác — dữ liệu có nội dung, exposure gần đầy đủ, hoặc split ngẫu nhiên đã xử lý sẵn — nên chỉ dùng làm bối cảnh, chưa chạy thực nghiệm.

3.2 Chất lượng và nhiễu

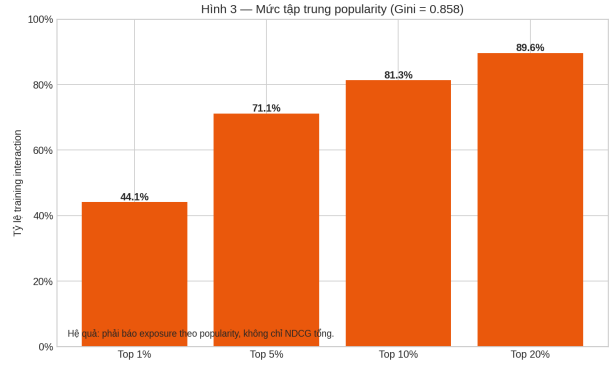
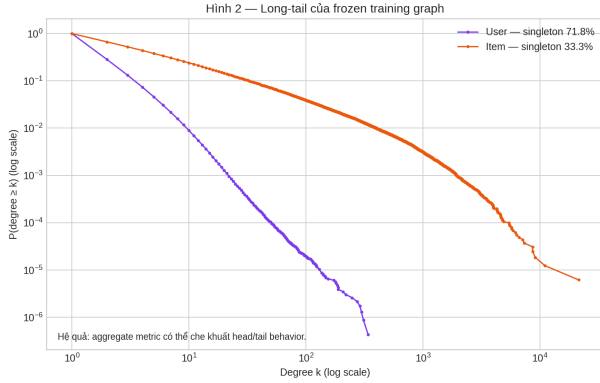
Kiểm toán Baby_Products cho thấy không có dòng thiếu ID, không có timestamp sai và không có cặp user–item trùng. Chỉ có một dòng rating 0.0 ngoài miền 1–5; dòng này bị loại. Phân bố rating lệch mạnh về 5 sao (Bảng 4). Quy tắc P4 xem rating 4–5 là tương tác positive, giữ 4.655.843 dòng (78,20%).

Không lọc k -core: lọc sẽ xóa đúng phần long-tail và user ít tương tác mà bài toán lấy mẫu cần đối mặt.

Bảng 4: Phân bố rating Baby_Products.

Rating	0	1	2	3	4	5
Số dòng	1	555.424	309.591	433.032	681.977	3.973.866

3.3 Mất cân bằng và long-tail

**Hình 1:** Trái: phân phối bậc user và item (log-log). Phải: tỷ lệ tương tác do item phổ biến nhất nắm giữ.

Graph huấn luyện rất thưa (mật độ $1,03 \times 10^{-5}$) và lệch (Hình 1):

- 71,76% user chỉ có một tương tác; bậc user $p_{50}/p_{90}/p_{99} = 1/3/9$.
- 33,35% item chỉ có một tương tác; bậc item $p_{50}/p_{90}/p_{99} = 3/32/397$; Gini bậc item 0,8584.
- Top 1% item giữ 44,09% tương tác; top 20% giữ 89,65%.

Hệ quả cho thiết kế: (i) điểm tổng có thể tăng chỉ nhờ phục vụ tốt item phổ biến, nên phải báo exposure và hit theo head/body/tail; (ii) user gần như không có ngữ cảnh riêng, nên lựa chọn của sampler có ý nghĩa chủ yếu ở layer đi qua item; (iii) sampler học được có thể dồn về hub, nên phải theo dõi bậc và nhóm của node được chọn.

3.4 Đường đi dữ liệu và chia theo thời gian

Pipeline: raw CSV (kiểm SHA-256) → kiểm toán → P4 → chia theo thời gian → graph train và mapping ID → huấn luyện có lấy mẫu → đánh giá full-graph, full-catalog.

Hai mốc hiện có là $t_1 = 11/08/2021$ và $t_2 = 16/07/2022$. Graph train chỉ gồm cạnh trước t_1 . Validation gồm 81.871 target warm-start trong $[t_1, t_2]$; test gồm 40.587 target sau t_2 và chưa được đọc. Vì validation đã bị pilot quan sát, giai đoạn 2 tạo một *development split* mới:

$$t_0 = t_1 - (t_2 - t_1), \quad G_{\text{dev_train}} = \{e : ts(e) < t_0\}, \quad D_{\text{dev}} = \{e : t_0 \leq ts(e) < t_1, \text{ warm-start}\}.$$

Nếu D_{dev} có dưới 20.000 target hoặc $G_{\text{dev_train}}$ mất quá 50% cạnh, dùng quy tắc dự phòng đã đăng ký: chọn t_0 sao cho $G_{\text{dev_train}}$ giữ 80% cạnh. Vai trò các split nêu ở Bảng 5.

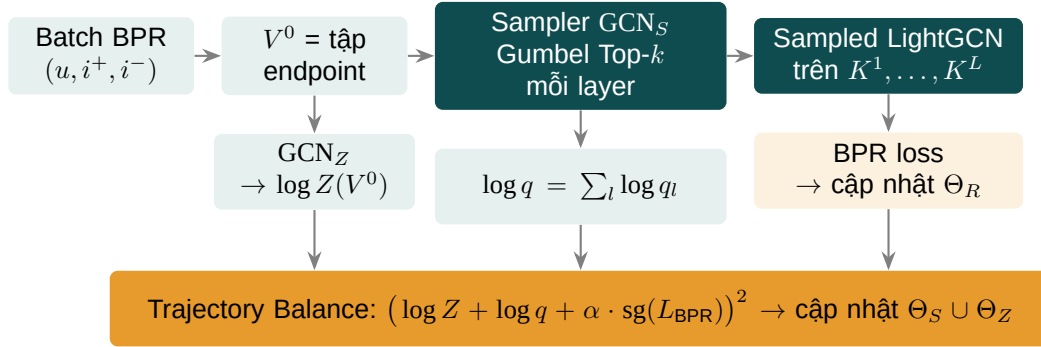
Bảng 5: Vai trò các split.

Split	Dùng cho	Cấm
$G_{\text{dev_train}} / D_{\text{dev}}$	Chọn budget, α , learning rate, kiến trúc sampler; kiểm tra backbone	Kết luận cuối
Graph train / validation	So sánh ghép cặp sau khi freeze cấu hình	Mọi điều chỉnh
Test	Một lần, nếu có protocol final-test riêng	Chọn phương pháp

Cạnh tương lai không được đi vào graph, degree, tập ứng viên, feature của sampler, normalization hay hàm mục tiêu.

4 Phương pháp đề xuất: GRAPES-GFN-Rec

4.1 Kiến trúc tổng thể



Hình 2: Một bước huấn luyện. sg là stop-gradient. Suy luận không lấy mẫu: LightGCN chạy trên toàn graph train, xếp hạng toàn catalog.

Hình 2 là khung kiến trúc chung. Sampler và recommender có tham số, optimizer và gradient tách biệt. Backbone có thể thay (ví dụ NGCF, SGL) mà không đổi sampler; trong triển khai thực tế, sampler chỉ dùng lúc huấn luyện, phục vụ gợi ý dùng embedding đã học.

4.2 Chuyển đổi từ GRAPES sang gợi ý

Bảng 6: Chuyển đổi thành phần. Các dòng bên phải là quyết định thiết kế của đồ án, không phải kết quả đã được paper GRAPES chứng minh.

Khía cạnh	GRAPES gốc	GRAPES-GFN-Rec
Đồ thị	Node có feature	Hai phía user-item, chỉ positive trước mốc thời gian
Mục tiêu batch	Node có nhãn	Triplet BPR; V^0 = unique endpoint, giữ ánh xạ triplet
Feature sampler	Feature node	Embedding ID riêng, one-hot loại node, $\log(1 + \text{deg})$ chuẩn hóa, cờ lịch sử $L+1$ kênh
Model chính	GCN phân loại	Sampled LightGCN: không self-loop, block $K^l \rightarrow K^{l-1}$, bi-normalization chữ nhật
Chi phí	Cross-entropy	Mean BPR loss, không gồm regularizer
$\log Z$	GCN trên target và hàng xóm	GCN_Z chỉ trên V^0
Đánh giá	Accuracy	Xếp hạng toàn catalog, NDCG@20, Recall@20, coverage, head/tail

4.3 Lấy mẫu theo layer

Với mỗi layer $l = 1, \dots, L$, tập ứng viên là hàng xóm của trạng thái trước trừ chính nó:

$$C^l = \mathcal{N}_{G_{\text{train}}}(K^{l-1}) \setminus K^{l-1}, \quad K^0 = V^0.$$

GCN_S hai lớp chạy trên subgraph cảm sinh bởi $K^{l-1} \cup C^l$ và cho logit ℓ_v , xác suất $p_v = \sigma(\ell_v)$ cho mỗi $v \in C^l$. Hành động chọn đúng min($k_l, |C^l|$) node:

$$V^l = \text{TopK}_{v \in C^l}(\log p_v + g_v), \quad g_v \sim \text{Gumbel}(0, 1), \quad K^l = V^0 \cup V^l.$$

Log-likelihood dùng phân phối Bernoulli đầy đủ, gồm cả node được chọn ($m_v = 1$) và không được chọn:

$$\log q_l = \sum_{v \in C^l} [m_v \log p_v + (1 - m_v) \log(1 - p_v)], \quad \log q = \sum_{l=1}^L \log q_l.$$

Hành động lấy từ phân phối exact- k trong khi likelihood là Bernoulli không điều kiện: đây là sai lệch off-policy kế thừa từ GRAPES và sẽ được nêu rõ.

Hai đối chứng M0 và M1 dùng cùng đường code, chỉ khác điểm: M0 dùng điểm bằng nhau, M1 dùng $\log \deg(v)$; cả hai không có tham số và $\log q = 0$. Nhờ vậy khác biệt giữa các sampler chỉ nằm ở cách chấm điểm.

4.4 Sampled LightGCN

Block $P_l \in \mathbb{R}^{|K^{l-1}| \times |K^l|}$ chứa cạnh gốc giữa nguồn K^{l-1} và đích K^l , chuẩn hóa hai phía theo bậc cục bộ trong block:

$$P_l[t, s] = \frac{B_l[t, s]}{\sqrt{d_l^{\text{dst}}(t) d_l^{\text{src}}(s)}}.$$

Embedding ở độ sâu r của V^0 là tích tiền tố $e^r = P_1 P_2 \cdots P_r E[K^r]$, và biểu diễn cuối là trung bình đều $z = \frac{1}{L+1} \sum_{r=0}^L e^r$.

Giới hạn đã được chứng minh bằng test. Vì K^l không cộng dồn, khi V^0 chứa cả user lẫn item, một số node thuộc V^1 bị loại khỏi K^2 ; do đó kể cả khi lấy hết ứng viên, kết quả không bằng Full LightGCN. Với target chỉ gồm một user, hai cách cho kết quả bằng nhau. Đây là tính chất của cách định nghĩa trạng thái trong GRAPES, không phải lỗi cài đặt.

4.5 Hàm mục tiêu

Recommender tối ưu

$$L_{\text{BPR}} = -\frac{1}{M} \sum_{b=1}^M \log \sigma(z_{u_b}^\top z_{i_b^+} - z_{u_b}^\top z_{i_b^-}), \quad L_{\text{model}} = L_{\text{BPR}} + \lambda L_{\text{reg}}.$$

Sampler tối ưu Trajectory Balance [3] với phần thưởng $R = \exp(-\alpha L_{\text{BPR}})$:

$$L_{\text{TB}} = \left(\log Z(V^0) + \log q + \alpha \cdot \text{sg}(L_{\text{BPR}}) \right)^2.$$

$\log Z(V^0)$ là trung bình đầu ra của GCN_Z trên subgraph cảm sinh bởi V^0 (có self-loop), nên bất biến với hoán vị và với node ngoài V^0 . Biến thể GRAPES-RL-Rec thay L_{TB} bằng $\text{sg}(L_{\text{BPR}}) \cdot \log q$.

Thuật toán 1: Một bước huấn luyện GRAPES-GFN-Rec

Input: batch triplet B , graph G_{train} , budget $(k_1, \dots, k_L)$, α

- 1 $V^0 \leftarrow \text{unique}(B)$; giữ ánh xạ triplet $\rightarrow$ chỉ số;
- 2 **for** $l \leftarrow 1$ **to** L **do**
- 3 $C^l \leftarrow \mathcal{N}(K^{l-1}) \setminus K^{l-1}$; $\ell \leftarrow \text{GCN}_S(K^{l-1} \cup C^l)$;
- 4 $V^l \leftarrow \text{GumbelTopK}(\log \sigma(\ell), k_l)$; $\log q \leftarrow \log q + \log q_l$; $K^l \leftarrow V^0 \cup V^l$;
- 5 $z \leftarrow \text{SampledLightGCN}(P_1, \dots, P_L)$; tính L_{model} ;
- 6 Cập nhật Θ_R bằng ∇L_{model} ; // sampler không nhận gradient này
- 7 $L_{\text{TB}} \leftarrow (\log Z(V^0) + \log q + \alpha \text{sg}(L_{\text{BPR}}))^2$;
- 8 Cập nhật $\Theta_S \cup \Theta_Z$ bằng ∇L_{TB} ; // recommender không nhận gradient này
- 9 Ghi log: $L_{\text{BPR}}, L_{\text{TB}}, \log q, \log Z, |C^l|, |V^l|$, bậc/loại/nhóm của V^l , thời gian, bộ nhớ;

4.6 Những điểm riêng của bài toán gợi ý

R-1 Quá ít phần thưởng.

Pilot dùng batch 65.536 trong 5 epoch, khoảng 300 bước tối ưu: sampler chỉ nhận khoảng 300 phần thưởng vô hướng cho hàng trăm nghìn node mục tiêu mỗi bước. GRAPES gốc dùng batch 256–512. Batch và budget sẽ được chọn lại trên development split, chung cho mọi sampler.

R-2 Thang phần thưởng.

BPR khởi đầu khoảng 0,69 và chênh lệch giữa trajectory rất nhỏ; nếu α quá nhỏ, $\log q$ lấn át phần thưởng. GRAPES gốc chọn hệ số trong khoảng 10^3 đến 10^5 . α và $\log Z$ khởi đầu được quét trên D_{dev} .

R-3 Phần thưởng không dừng.

Khi recommender học thêm, cùng trajectory cho loss khác. Giữ như GRAPES và ghi log mức trôi.

R-4 Graph cực thưa.

Theo dõi phân phối bậc, loại và nhóm của node được chọn để phát hiện sampler dồn về hub.

R-5 Chi phí sampler.

Embedding cho 2,48 triệu node cộng GCN_S có thể đắt hơn LightGCN; chi phí báo cáo gồm cả sampler.

R-6 Backbone yếu.

Trong pilot, Full LightGCN (NDCG@20 0,0049) thua MostPop (0,0059). Full LightGCN phải vượt MostPop trên D_{dev} trước khi so sánh sampler có ý nghĩa.

R-7, R-8.

Item âm cũng là mục tiêu của sampler; cạnh positive của batch có thể rò qua propagation, được kiểm tra bằng ablation che cạnh tạm thời.

5 Thiết kế đánh giá

5.1 Phương pháp so sánh

Bảng 7: Các phương pháp so sánh.

Tầng	Phương pháp	Vai trò	Câu hỏi trả lời
A	GRAPES-GFN-Rec	Phương pháp đề xuất	–
A	M0 Uniform	Đối chứng trung lập	Học có hơn chọn ngẫu nhiên?
A	M1 Degree-importance	Đối chứng tình (họ FastGCN/LADIES [6, 7])	Học có hơn ưu tiên node phổ biến?
A	GRAPES-RL-Rec	Ablation hàm mục tiêu	TB có cần hơn REINFORCE?
B	Full LightGCN	Không lấy mẫu	Lấy mẫu mất bao nhiêu chất lượng?
B	MostPop, BPR-MF	Sanity	Graph có giá trị?
C	GraphSAGE fan-out [8]	Tùy chọn	Layer-wise học được vs node-wise?

Tầng A dùng chung graph, thứ tự triplet, negative, khởi tạo, budget, seed, evaluator và GPU, và được chạy lại hoàn toàn; số liệu pilot không được dùng lại.

5.2 Chỉ số

- **Chất lượng:** NDCG@20 (chính), Recall@20; chênh lệch ghép cặp theo seed.
- **Phân bố và đa dạng:** Catalog Coverage@20; exposure và hit theo head/body/tail; Gini exposure.
- **Chi phí:** thời gian sampler, TB, propagation và epoch; peak GPU memory; số node/cạnh thực tế mỗi layer.
- **Hành vi sampler:** bậc, loại, nhóm của node được chọn; entropy; $p_{\min}, p_{\max}$; độ thay đổi tham số sampler.
- **Ngữ nghĩa:** dữ liệu hiện tại chỉ có ID, rating và thời gian, nên mức khớp ngữ nghĩa và đa dạng sản phẩm chưa đo được. Có thể nối metadata Amazon (title, category) để đo đa dạng theo danh mục như phân tích phụ. Conversion không đo được vì dữ liệu offline không có log hiển thị.

5.3 Tiêu chí kết luận

Chỉ kết luận GRAPES-GFN-Rec cải thiện trade-off khi chênh lệch NDCG@20 so với đối chứng tăng A tốt nhất dương ở mọi seed holdout và chi phí được báo đầy đủ. Một run không hợp lệ nếu vi phạm cách ly thời gian, exact- k , quy tắc ứng viên, hướng block và normalization, tính hữu hạn của loss, phân quyền gradient, khả năng lặp lại hoặc thiếu log tài nguyên. Không đổi phương pháp, α , budget hay seed sau khi đọc holdout.

6 Tiến độ hiện tại

- **R0 — khóa scope:** spec chuẩn, decision log DL-001, checklist G1–G7, công cụ kiểm tra tính nhất quán từ chối mô tả M2 là phương pháp giai đoạn 2.
- **R2 — lỗi mã nguồn:** sampler (learned, uniform, degree), Sampled LightGCN, bước huấn luyện TB/REINFORCE. 28 oracle test pass trên PyTorch CPU, gồm: tập ứng viên khớp brute-force; exact- k ; lấy mẫu đều khi điểm bằng nhau; log q khớp tính tay; đường đi mẫu cho giá trị 1 (không chuẩn hóa), $1/2$ (bậc toàn graph), $1/\sqrt{2}$ (bậc cục bộ); TB khớp tính tay; log Z bất biến hoán vị; TB không tạo gradient cho recommender và BPR không tạo gradient cho sampler; tham số sampler thay đổi sau một bước; lặp lại xác định.
- **Kiểm tra độ nhạy của test:** bốn lỗi được cài cố ý (bỏ detach trong TB, bỏ số hạng node không chọn trong log q , cộng dồn K^l , sai dấu REINFORCE) đều bị bộ test phát hiện.

7 Kế hoạch triển khai

Bảng 8: Các gate của giai đoạn 2. Thời lượng là ước lượng, sẽ chỉnh sau R3.

Gate	Nội dung	Điều kiện qua gate	Tuần	Trạng thái
R0	Khóa scope, spec, decision log, checker	Không tài liệu trung tâm gọi M2 là phương pháp	0,5	Xong
R2	Sampler, Sampled LightGCN, TB, oracle test	Toàn bộ oracle test pass	1	Lỗi xong
R1	Development split, manifest, hash (Colab)	Kiểm tra cách ly thời gian pass	0,5	Tiếp theo
R3	Backbone, budget, quét α/lr , khả thi T4, sampler học thật	Full LightGCN > MostPop trên D_{dev} ; loss hữu hạn; lặp lại được	3	
R4	Freeze cấu hình, seed, evaluator, script phân tích	Manifest tạo trước khi đọc validation	0,5	
R5	So sánh ghép cặp trên holdout	Hash khớp; metric tính lại được	2	
R6	Phân tích RQ1–RQ4, ablation	Mọi bảng truy về JSON	1	
R7	Báo cáo, slide, README; pilot vào phụ lục	Kiểm tra nhất quán pass	1,5	

Việc còn lại của R2 gồm: kiểm tra negative hợp lệ, ablation che cạnh positive, ablation log Z kiểu cũ và cách lưu embedding thừa cho 2,48 triệu node.

8 Rủi ro và phương án dự phòng

Rủi ro	Dấu hiệu	Phương án
Sampler không học	Tham số gần như không đổi; phân phối chọn giống M0	Tăng số bước, giảm batch, quét α ; nếu vẫn không học, báo kết quả âm có phân tích
Hết bộ nhớ T4	OOM với embedding sampler	Embedding thưa, giảm chiều, chỉ chạy GCN _S trên ứng viên
Sampler quá chậm	Vượt ngưỡng thời gian đăng ký	Giảm độ rộng GCN _S , lấy mẫu ứng viên trước khi chấm điểm
Backbone vẫn yếu	Full LightGCN $\leq$ MostPop trên D_{dev}	Tăng epoch trong grid; nếu fail ghi decision log trước khi đi tiếp
Dồn về item phổ biến	Node chọn toàn head, tail hit bằng 0	Báo cáo trong RQ3; không đổi phương pháp sau holdout

9 Quyết định cần ý kiến giảng viên

OD-1: Có thêm GraphSAGE fan-out làm đối chứng tầng C không? Mặc định: không.

OD-2: Có nổi metadata sản phẩm để đo đa dạng và mức khớp ngữ nghĩa không? Mặc định: không trong so sánh chính.

OD-3: Lưới budget cho RQ4? Mặc định: một budget chính và một budget chặt.

OD-4: Số seed holdout? Mặc định: 3.

Tài liệu

- [1] T. Younesian et al. *GRAPES: Learning to Sample Graphs for Scalable Graph Neural Networks*. arXiv:2310.03399v3. Mã nguồn: <https://github.com/dfdazac/grapes>, commit 71ecebe.
- [2] X. He et al. *LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation*. SIGIR 2020. arXiv:2002.02126.
- [3] N. Malkin et al. *Trajectory Balance: Improved Credit Assignment in GFlowNets*. NeurIPS 2022. arXiv:2201.13259.
- [4] S. Rendle et al. *BPR: Bayesian Personalized Ranking from Implicit Feedback*. UAI 2009. arXiv:1205.2618.
- [5] W. Kool et al. *Stochastic Beams and Where to Find Them: The Gumbel-Top-k Trick for Sampling Sequences Without Replacement*. ICML 2019. arXiv:1903.06059.
- [6] J. Chen et al. *FastGCN: Fast Learning with Graph Convolutional Networks via Importance Sampling*. ICLR 2018. arXiv:1801.10247.
- [7] D. Zou et al. *Layer-Dependent Importance Sampling for Training Deep and Large Graph Convolutional Networks*. NeurIPS 2019. arXiv:1911.07323.
- [8] W. Hamilton et al. *Inductive Representation Learning on Large Graphs*. NeurIPS 2017. arXiv:1706.02216.
- [9] R. Ying et al. *Graph Convolutional Neural Networks for Web-Scale Recommender Systems*. KDD 2018. arXiv:1806.01973.
- [10] Y. Hou et al. *Bridging Language and Items for Retrieval and Recommendation*. arXiv:2403.03952. Dữ liệu: <https://amazon-reviews-2023.github.io/>.